Pontificia Universidad Católica de Chile
Escuela de Ingeniería
Departamento de Ingeniería Eléctrica
IEE2463 Sistema electrónicos programables

Ayudantia 09 2S26

Protocolo UART

Ayudante: Ernesto Ferrante ernesto.ferrante@uc.cl
Prof.Ing. Michel Rozas - mvrozas@uc.cl

---

1. Objetivo de la Ayudantía

- Habilitar terminal en vitis o externa para comunicarse mediante UART entre PC-ZYNQ y ZYNQ-PC.
- ejercitar el uso de estructuras, punteros y funciones en un caso práctico.

2. Actividades Durante la Ayudantía: Máquina Expendedora

En esta actividad se programará en Vitis una máquina expendedora que se controla completamente desde la consola serial (UART). El usuario podrá ver los productos disponibles, comprar uno ingresando su código y su pago, y recibir el vuelto desglosado en monedas. Los LEDs de la Zybo indicarán qué producto se está entregando. Además, existirá un modo administrador protegido con clave para reponer el stock.

Se utilizará el mismo hardware de las ayudantías anteriores: un MicroBlaze con un AXI GPIO conectado a los 4 LEDs (canal 1) y la UART configurada como entrada y salida estándar. La consola debe abrirse con el baud rate configurado en el diseño, 8 bits de datos, sin paridad y 1 bit de parada (8N1).

2.1. Funciones útiles

Función	Descripción
`xil_printf("...")`	Imprime texto por la UART. No soporta números decimales (`%f`). Usar `\r\n` como salto de línea.
`scanf("%d", &x)`	Lee un número entero desde la consola. Espera a que se presione Enter.
`scanf("%3s", texto)`	Lee una palabra de máximo 3 caracteres. Al ser un arreglo, `texto` se pasa sin `&`.
`strcmp(a, b)`	Compara dos strings. Retorna 0 si son iguales.
`inbyte()`	Lee **un solo** carácter desde la UART, sin esperar Enter y sin hacer eco.
`outbyte(c)`	Imprime **un solo** carácter por la UART.
`usleep(t)`	Detiene la ejecución durante `t` microsegundos.
`XGpio_DiscreteWrite()`	Escribe `valor` en los LEDs (cada bit es un LED).

2.2. Actividades

1. **Estructura y datos.** Defina una estructura `Producto` con los campos `codigo` (string de largo 4), `nombre` (string de largo 16), `precio` y `stock` (enteros). Cree un arreglo global de 4 productos con los siguientes datos:

Código	Nombre	Precio	Stock
A1	Agua	$800	3
A2	Bebida	$1200	2
B1	Galletas	$950	1
B2	Chocolate	$1000	0

   Implemente la función `void mostrarProductos(void)`, que imprime en la consola el código, nombre, precio y stock de cada producto.

2. **Búsqueda de productos.** Implemente la función `int buscarProducto(char *codigo)`, que recorre el arreglo y retorna el índice del producto cuyo código coincide con el ingresado, o `-1` si no existe.

3. **Compra.** Implemente la función `void comprar(void)`, que:

   - Pide el código del producto y lo busca con `buscarProducto`.
   - Muestra un mensaje de error si el código no existe o si el producto está agotado.
   - Muestra el precio y pide el pago. Si el pago no alcanza, informa que se devuelve el dinero.
   - Si el pago es suficiente, descuenta una unidad del stock, entrega el producto (actividad 5) y calcula el vuelto (actividad 4).

4. **Vuelto.** Implemente la función `void darVuelto(int vuelto)`, que muestra el vuelto total y su desglose usando la menor cantidad posible de monedas de $500, $100, $50 y $10. Por ejemplo, un vuelto de $850 corresponde a 1 moneda de $500, 3 de $100 y 1 de $50.

   Pista: la división entera (`/`) indica cuántas monedas de un valor caben, y el módulo (`%`) indica cuánto queda por pagar.

5. **Entrega con LEDs.** Implemente la función `void entregar(int i)`, que hace parpadear 5 veces el LED correspondiente al producto `i` (250 ms encendido y 250 ms apagado). En cada parpadeo, imprima un punto con `outbyte` para formar una animación del tipo `Entregando Agua ..... listo!`

   Pista: la expresión `1 << i` genera un número con solo el bit `i` en 1.

6. **Modo administrador.** Implemente la función `void reponer(void)`, que:

   - Pide una clave y la compara con `"1234"`. Si es incorrecta, vuelve al menú.
   - Pide el código y la cantidad a reponer **en una misma línea** (por ejemplo, `B2 5`) usando un solo `scanf`.
   - Valida que el producto exista y que la cantidad sea positiva, suma la cantidad al stock y muestra el nuevo stock.

7. **Menú principal.** En el `main`, inicialice el GPIO, apague los LEDs y cree un ciclo infinito que muestre los productos y el menú. La opción se debe leer con `inbyte()`, de modo que basta presionar la tecla `1` (comprar) o `2` (reponer) sin Enter, como si fueran los botones de la máquina. Cualquier otra tecla debe mostrar `Opcion invalida`. Considere que `inbyte()` también puede leer los caracteres `\r` y `\n` de un Enter anterior; estos deben ser ignorados.

2.3. Ejemplo de ejecución

La siguiente es una posible interacción con el programa terminado. Los valores `A2` y `2000` son los ingresados por el usuario.

```
===== Maquina expendedora =====
A1 - Agua ($800) stock: 3
A2 - Bebida ($1200) stock: 2
B1 - Galletas ($950) stock: 1
B2 - Chocolate ($1000) stock: 0
Presione 1 para comprar o 2 para reponer
Ingrese el codigo del producto:
A2
Precio: $1200. Ingrese su pago:
2000
```

```
Entregando Bebida ..... listo!
Vuelto: $800
  1 x $500
  3 x $100
```

2.4. Desafíos opcionales

- Mostrar en los LEDs, en binario, el stock del último producto comprado.
- Bloquear el modo administrador durante 10 segundos después de 3 claves incorrectas.
- Agregar una opción `3` que muestre el total de dinero recaudado por la máquina.